

Exercices

Smart Pointers - Par notions

1 Notion : `unique_ptr` (propriete exclusive)

1.1 Exercice 1 : Convertir raw pointers en `unique_ptr` (facile)

Enonce

Convertissez ce code utilisant des raw pointers en code utilisant `unique_ptr` :

```
class Data {
    int value;
public:
    Data(int v) : value(v) {}
    int getValue() const { return value; }
};

int main() {
    Data* d1 = new Data(10);
    Data* d2 = new Data(20);

    std::cout << d1->getValue() << "\n";
    std::cout << d2->getValue() << "\n";

    delete d1;
    delete d2;
}
```

1.2 Exercice 2 : Gestionnaire de ressources (moyen)

Enonce

Creez une classe `ResourceManager` qui :

- Stocke des ressources (classe `Resource`) avec `unique_ptr`
- Methode `addResource(nom, unique_ptr)` pour ajouter
- Methode `getResource(nom)` qui retourne un pointeur brut
- Methode `releaseResource(nom)` qui retire et retourne le `unique_ptr`

2 Notion : `shared_ptr` (propriete partagee)

2.1 Exercice 3 : Arbre avec `shared_ptr` (difficile)

Enonce

Creez une structure d'arbre binaire ou :

- Chaque noeud a deux enfants (gauche, droite) avec `shared_ptr`
- Chaque noeud a un parent avec `weak_ptr`
- Methode `insert(valeur)` pour inserer
- Methode `contains(valeur)` pour chercher
- Methode `afficherParent()` pour afficher le parent d'un noeud